Digital Systems Project Assignment 1
Digital Watch: Settable Time

https://chipverify.com/verilog/verilog-data-types

What values do variables hold ?

Almost all data-types can only have one of the four different values as given below except for `real` and `event` data types.

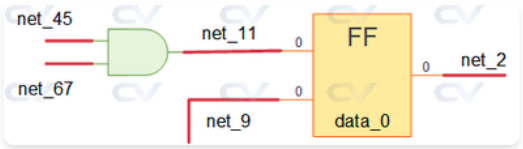
0	represents a logic zero, or a false condition
1	represents a logic one, or a true condition
x	represents an unknown logic value (can be zero or one)
z	represents a high-impedance state

Nets and Variables

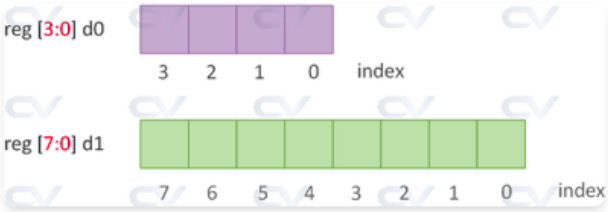
Nets and *variables* are the two main groups of data types which represent different hardware structures and differ in the way they are assigned and retain values.

Nets

Nets are used to connect between hardware entities like logic gates and hence do not store any value on its own. In the image shown below, a net called `net_11` is used to connect between the output of the AND gate to the first input of the flip-flop called `data_0`. In a similar way, the two inputs of the AND gate are connected to nets `net_45` and `net_67`.



Variables



A variable on the other hand is an abstraction of a data storage element and can hold values. A flip-flop is a good example of a storage element.

https://chipverify.com/systemverilog/systemverilog-data-type-logic

logic: A 4-State Data Type

`logic` is a 4-state data type — its value set (`0` , `1` , `X` , `Z`) is identical to `reg` 's. It isn't, by itself, a variable or a net; it's just the type. When you write a plain declaration like `logic a;` with no other keyword, SystemVerilog implicitly treats that as `var logic a;` — a **variable** of type `logic` .

5. Development Cycle

The golden rule of software development — which applies equally to Verilog development — is to write a *small* amount of code, compile and test it thoroughly, finalise its documentation, and then leave it alone unless you need to fix a bug down the track. We use this approach to build small, reusable modules. Near the end, when these modules must be assembled, we shift to a feature-layered strategy: add a small feature, compile and test thoroughly, then move on to the next feature.

Thousands of lines of automated test code have been written for you. In real life, you or your team would be responsible for developing and maintaining this test code. As the saying goes, *"if it is not tested, it is broken."* Test code is critical: it improves reliability, prevents costly defects, makes changes safer, and serves as living documentation.

Each write-compile-test loop follows what we call the *development cycle*. This cycle is exemplified below.

Conventions A good project has an agreed set of conventions.

Each module should live in its own file within the `rtl/` subdirectory, with the filename matching the module name. Names should be descriptive and should only be abbreviated when the full name would be overly long. Well-established abbreviations such as `clk` and `rst` are acceptable, provided they are used consistently.

Naming conventions are as follows. (Commonly, `localparam` constants are also capitalised, but our particular linter disagrees.)

Kind	Convention	Example
Modules and signals	`snake_case`	`next_state`, `tick_enable`
`parameter` constants	`ALL_CAPS`	`MAX`, `WIDTH`
`localparam` constants	`CamelCase`	`TicksPerSecond`

5.1. Module Specification

For each module, you will be given its declaration together with a description of the required behaviour. Follow the specification exactly, or the automated tests will fail.

5.2. Write the Module

Whenever you are asked to write a new module — starting now, for the `seven_segment` specification above — do the following:

5.3. Lint, Compile then Generate Waveforms

In this assignment we use the terminal for four things (in the next assignment, it will also be used to run a formal verification tool):

1. More thorough linting (`verilator -Wall --lint-only`).
2. Compiling code (`iverilog -Wall -g2012`).
3. Running test benches (`iverilog -Wall -g2012`, then `./a.out`).
4. Running the automated test suite (`pytest`).

Lint Run: `verilator -Wall --lint-only rtl/seven_segment.sv`

It reports 20 warnings before stopping. These are warnings rather than errors: the code is valid SystemVerilog and will compile. **Do not mistake "warning" for "harmless": a warning usually means the tool has noticed you doing something you did not intend.** Most of the warnings below correspond to genuine bugs. Even the one that does not, noted as it arises, should still be eliminated: being explicit about your intent is good practice, and the goal is always zero warnings.

Report 1

Explain why Verilator accepts `if (blank)` but not `if (ACTIVE_LOW)`.

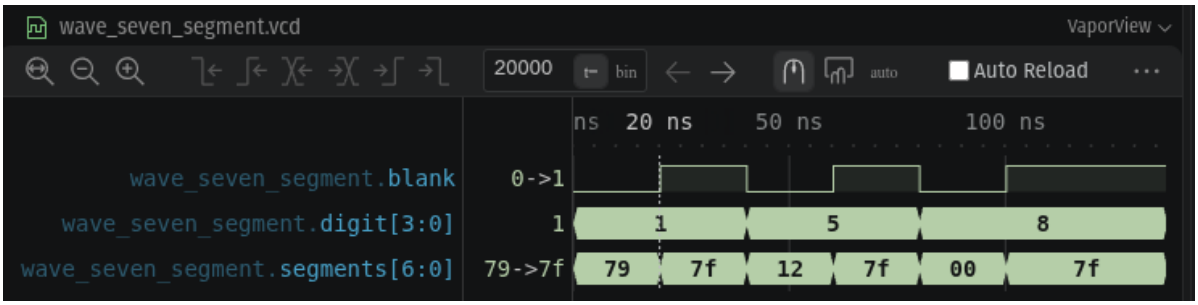
```
%Warning-WIDTHTRUNC: rtl/seven_segment.sv:42:32: Logical operator COND expects 1 bit on
the Conditional Test, but Conditional Test's VARREF 'ACTIVE_LOW' generates 32 bits.
                                                : ... note: In instance 'seven_segment'
   42 |   assign segments = ACTIVE_LOW ? !segments_active_high : segments_active_high;
      |                                ^
```

The if-conditional statement requires its expression to resolve to either True (1) or False (0).
In other words, a binary outcome from 1-bit. It does not encode three or more outcomes by itself.

ACTIVE_LOW is declared as an `int`, which is a 2-state, 32-bit signed integer, and it can contain values other than 0 or 1. Although any non-zero values gets treated as True, the if condition still expects only single-bits and not 32-bits for the conditional tests



```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only rtl/seven_segment.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/seven_segment.sv
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/seven_segment.sv tb/wave_seven_segment.v
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_seven_segment.vcd opened for output.
tb/wave_seven_segment.v:41: $finish called at 120000 (1ps)
root@4034a70d84cc:/workspace#
```



5.4. Run the Automated Tests

From the terminal, run `pytest tests/test_seven_segment.py` to test only this module, or `pytest` to test all modules. Tests for unwritten modules will be skipped.

Check that all tests for `tests/test_seven_segment.py` pass. If they are reported as skipped, the file is either misnamed or not in the `rtl` subdirectory.

If any tests fail, work through the following steps:

- Scroll up through the `pytest` output to identify which test failed.
- Read the comments in `tests/cocotb/tb_seven_segment.py` to understand what the failing test checks.
- Focus on the corresponding part of your module.

If you are still stuck, see the next section.

```
root@4034a70d84cc:/workspace# pytest tests/test_seven_segment.py
========================== test session starts ==========================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 2 items

tests/test_seven_segment.py ..                                    [100%]

=========================== 2 passed in 0.57s ===========================
root@4034a70d84cc:/workspace#
```

5.5. Write Your Own Test Benches

If your module is still not working, write your own test bench. Although the error dump from `pytest` is verbose, and much of it may be unintelligible, scroll through it for messages that make sense. There will usually be a comment explaining what the test was checking, followed by a statement of why it failed. Target that same behaviour in your test bench: your aim is to reproduce the fault.

If reproducing the faulty behaviour requires many steps, simplify it to the fewest steps that still exhibit the fault. Focus on one bug at a time.

Once you have a test bench that reproduces the bug, trace your code through the same steps. This should reveal where your code behaves unexpectedly.

5.6. Fulfil Report and Oral Preparation Obligations

The following obligations apply to every module, including `seven_segment`.

Report 2

For every module, include a screenshot of the waveforms and briefly explain why they are correct, pointing to specific features in the traces.

Additional Oral Preparation 1

For every module, be able to describe its behaviour precisely for all input combinations.

Additional Oral Preparation 2

Be able to explain the development cycle and the project conventions in detail.

5.7. Backup

After each module passes all its tests, back up your code and report.

Report 3

At the start of your report, describe your backup strategy.

- Fork Repo
- Push commits

6. Time Display

By the end of this section, the seven-segment displays will show hours, minutes, and seconds, incrementing once per second like a clock. Switches select faster rates. Time is in 24-hour format.

6.1. State Encoding

Before writing any code, we must decide how to represent the current time in hardware. There are at least four reasonable choices.

1. **One counter (clock cycles).** Count the total number of rising edges since midnight. With a 50 MHz clock, a rising edge occurs every 20 ns.
2. **One counter (seconds).** Count the total number of seconds elapsed since midnight.
3. **Three counters.** Maintain separate counters for hours (0–23), minutes (0–59), and seconds (0–59).
4. **Six counters.** Maintain a separate counter for each display digit: the tens and units of hours, minutes, and seconds.

The choice of state encoding directly influences the number of flip-flops required, the complexity of the next-state logic, and the complexity of the output logic.

The encoding choice also influences the time-setting hardware developed later. Time-setting works by cycling through each unit (seconds, minutes, and hours) and letting the user increment or decrement each independently.

This project adopts the three-counter encoding: separate counters for hours, minutes, and seconds.

Additional Oral Preparation 3

Be able to explain all four state encoding options and the trade-offs between them, and justify why the three-counter encoding was chosen.

1 Day = 24 Hours = 1440 Minutes = 86400 Seconds

Option 1: One Counter (Clock Cycles)
CLOCK_50 is 50MHz clock, this means if the Counter is counting total number of rising edges since midnight, there are (50 x 10^6) * (86400) rising edges = 4.32 x 10^12 rising edges.
Therefore, the Counter needs log_2_(4.32x10^12) = 41.97 = 42-bit length Rising-Edge Counter
Total FlipFlops required = 42

Option 2: One Counter (Seconds)
Counter requires a Modulo-50MHz counter to add 1 to itself. Max value is 86400 (seconds).
Seconds Counter needs log_2_(86400) = 16.40 = 17-bit length Seconds Counter
Total FF required = 17
Requires continuous assignment logic to factor states of Seconds Counter into required HH:MM:SS.

Option 3: Three Counters (Hours, Minutes and Seconds)
Separate Counters for Hours (0-23), Minutes (0-59) and Seconds (0-59).
- Seconds Counter needs Modulo-60 to reset to 0 and adds 1 to Minutes Counter.
-- Needs log_2_(59) = 5.88 = 6-bit length Seconds Counter.
- Minutes Counter similar to Seconds Counter: Modulo-60 to reset to 0 and adds 1 to Hours Counter.
-- Needs log_2_(59) = 5.88 = 6-bit length Minutes Counter.
- Hours Counter needs Modulo-24 counter and resets to 0 for next day.
-- Needs log_2_(23) = 4.52 = 5-bit length Hours Counter.
- Total FF required = 6 + 6 + 5 = 17
- The 3 Counters encode the HH:MM:SS states directly. No need for additional logic

Option 4: Six Counters (Tens and Ones Counters for Hours, Minutes and Seconds)
Can use parametrised Counters, just need different values for each of the 6 Counters
Max 5/9 for Tens/Ones for Minutes and Seconds. FF = 4+4+4+4 = 16
Max 2/9 for Tens/Ones for Hours, with additional logic condition of 2/3. FF = 2+4 = 6
Total FF required = 16 + 6 = 22
Although only 2 parametrised modules are required (Tens and Minutes) to represent HH:MM:SS, additional logic is required to ensure HH:MM:SS properly loops back to 0, with HH having more.

In this assignment we use the terminal for four things (in the next assignment, it will also be used to run a formal verification tool):

1. More thorough linting (`verilator -Wall --lint-only`).
2. Compiling code (`iverilog -Wall -g2012`).
3. Running test benches (`iverilog -Wall -g2012`, then `./a.out`).
4. Running the automated test suite (`pytest`).

6.2. A Parametrised Up-Down Counter with Enable

You must follow the development cycle for every module (Section 5). Among other things, this includes eliminating all Verilator lint warnings and errors, generating and including waveforms in your report, and ensuring that all automated tests pass.

```
module up_down_counter #(
    parameter int MAX   = 2,
    parameter int WIDTH = 2
) (
    input logic clk,
    input logic enable,
    input logic up,
    output logic [WIDTH-1:0] count
);
```

The `count` is initialised to 0.

When `enable` is low, `count` does not change.

When `enable` is high, `count` increments if `up` is high and decrements if `up` is low, wrapping from `MAX` to 0 on increment and from 0 to `MAX` on decrement.

Rationale To edit time conveniently, we must be able to increment and decrement the count.

Implementation Notes It is conventional[1] to gate `enable` inside the `always_ff` block, even though it could instead be absorbed into the next-state logic.

```
always_ff @(posedge clk) if (enable) count <= next_count;
```

Use `always_comb` for the next-state logic.

Verilator will flag bit-width mismatches; use SystemVerilog's cast notation to adjust the width of a value explicitly.

[1] This mirrors real flip-flops, many of which have a dedicated clock-enable pin.

rtl > up_down_counter.sv > Verible > up_down_counter

```sv
// Up_Down_Counter
// To act as a standard Counter, whose modulus value is determined by the
// MAX parameter argument that was passed into it. The counter will count up
// when the up input is high, and count down when the up input is low. The
// counter will wrap back to 0 when the maximum value is reached, and wrap
// back to the maximum value when the minimum value is reached for up and
// down counting, respectively.
//
// Parameters:
//   MAX    - Maximum count value (default: 2).
//   WIDTH  - Width of the count output (default: 2).
//
// Ports:
//   clk    - Clock input.
//   enable - When high, the counter is enabled.
//   up     - When high, the counter counts up; when low, counts down.
//

`timescale 1ns / 1ps

module up_down_counter #(
    parameter int MAX   = 2,
    parameter int WIDTH = 2
) (
    input logic clk,
    input logic enable,
    input logic up,
    output logic [WIDTH-1:0] count = WIDTH'(0)
);

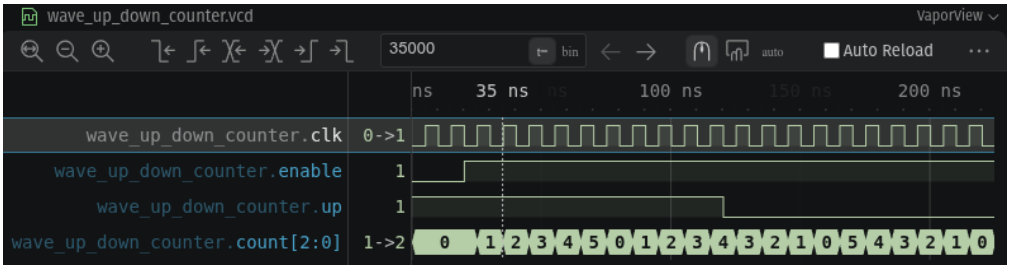
  // Declare local parameters and signals
  // Cast input `MAX` parameter to local `Max` parameter
  localparam logic [WIDTH-1:0] Max = WIDTH'(MAX);
  logic [WIDTH-1:0] next_count;

  // State, and Output Logic
  always_ff @(posedge clk) if (enable) count <= next_count;

  // Next state logic
  always_comb begin
    if (up) begin  // up=1 -> Count up
      if (count < Max) next_count = count + WIDTH'(1);
      else next_count = WIDTH'(0);  // Wrap to 0 when Max
    end else begin  // else up=0 -> Count down
      if (count > WIDTH'(0)) next_count = count - WIDTH'(1);
      else next_count = Max;  // Wrap to Max when 0
    end
  end

```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only rtl/up_down_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/up_down_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/up_down_counter.sv tb/wave_up_down_counter.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_up_down_counter.vcd opened for output.
tb/wave_up_down_counter.v:26: $finish called at 220000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_up_down_counter.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 3 items

tests/test_up_down_counter.py ...                                        [100%]

============================== 3 passed in 0.93s ===============================
root@4034a70d84cc:/workspace#
```

6.3. Binary to Binary-Coded Decimal

```
module binary_to_bcd (
    input  logic [6:0] bin,   // binary input, 0-99
    output logic [3:0] tens,  // decimal tens digit (BCD)
    output logic [3:0] ones   // decimal ones digit (BCD)
);
```

For any input value `bin` in the range 0 to 99, the output `tens` represents the decimal tens digit of `bin`, and the output `ones` represents the decimal ones digit.

Rationale Displaying hours, minutes and seconds on seven-segment displays requires a binary to binary-coded decimal converter.

> **Report 4**
>
> Explain what binary-coded decimal (BCD) is. You may need to research it.

Implementation Notes The linter expects the same bit width on both sides of a division. Since `bin` is 7 bits wide, we must write `bin / 7'd10`.

Writing `assign tens = bin / 7'd10` will still produce a warning: the LHS (left-hand side) is 4 bits wide, but the RHS (right-hand side) is 7 bits wide. As we genuinely want to discard the leading bits, we must explicitly convert the result to 4 bits.

```
assign tens = 4'(bin / 7'd10);
```

This may seem pedantic when you want to finish the assignment quickly, but in mission-critical designs, you must always be conscious of bit width. The linter forces you to be explicit about your intent.

When viewing the waveforms, set the viewer format for `bin` to decimal.

https://www.electronics-tutorials.ws/binary/binary-coded-decimal.html

What is a Binary Coded Decimal Number?

Binary Coded Decimal (BCD) is another type of binary encoding used in digital and electronic circuits. Unlike the standard binary numbering system, which converts the entire number, BCD converts each digit individually, making it ideal for 7-segment displays and input keyboards.

BCD For Driving Output Displays

Nevertheless, the use of a BCD coding system in both microelectronics and computer systems is particularly useful in situations where the binary coded decimal is intended to be displayed on one or more 7-segment LED or LCD displays and there are many popular integrated circuits available that are configured to give a BCD output or outputs.

Binary Coded Decimal Summary

We have seen here that **Binary Coded Decimal** or **BCD** is simply the 4-bit binary code representation of a decimal number. Each decimal digit replaced in the integer and fractional parts with its binary equivalent. BCD Code uses four bits to represent the 10 decimal digits of 0 to 9.

The main advantage of the **Binary Coded Decimal** system is that it is a fast and efficient system to convert the decimal numbers into binary numbers as compared to the pure binary system. But the BCD code is wasteful as many of the 4-bit states (10-to-16) are not used.

This because with 4 bits, we can represent sixteen numbers (0000 to 1111) in binary. However, in the BCD code only the first ten are used (0000 to 1001). The remaining six code combinations, i.e., 1010 to 1111 (A to F in Hex) are invalid in the binary coded decimal numbering format.

rtl > binary_to_bcd.sv > ...

```sv
// Binary to Binary-Coded Decimal (BCD) Converter
// Displaying hours, minutes, and seconds on seven-segement displays
// requires a binary to binary-coded decimal (BCD) converter.
// This module converts a 7-bit binary input (0-99) into two
// 4-bit BCD outputs representing the tens and ones digits.
//
// Note: Module allows conversion up to d'127 even though the
// maximum expected input is d'99. The BCD outputs will be d'12 and
// d'7 for an input of d'127.
//
// Parameters:
//   None
//
// Ports:
//   bin       [6:0]   - Binary input b'(0-110_0011) for d'(0-99).
//   tens      [3:0]   - BCD output for the tens digit.
//   ones      [3:0]   - BCD output for the ones digit.
//

`timescale 1ns / 1ps

module binary_to_bcd (
    input  logic [6:0] bin,   // binary input, b'(0-110_0011)
    output logic [3:0] tens,  // decimal tens digit (BCD)
    output logic [3:0] ones   // decimal ones digit (BCD)
);

  assign tens = 4'(bin / 7'd10);  // Calculate the tens digit
  assign ones = 4'(bin % 7'd10);  // Calculate the ones digit

endmodule

```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only rtl/binary_to_bcd.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/binary_to_bcd.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/binary_to_bcd.sv tb/wave_binary_to_bcd.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_binary_to_bcd.vcd opened for output.
tb/wave_binary_to_bcd.v:37: $finish called at 100000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_binary_to_bcd.py
========================= test session starts ==========================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_binary_to_bcd.py .                                    [100%]

========================== 1 passed in 0.31s ===========================
root@4034a70d84cc:/workspace#
```

6.4. Cascaded Hour-Minute-Second Counter with Enable

```
module hms_counter #(
    parameter int N_HOURS   = 24,  // number of hours
    parameter int N_MINUTES = 60,  // number of minutes
    parameter int N_SECONDS = 60,  // number of seconds

    // Output port widths
    parameter int W_HOURS   = 5,
    parameter int W_MINUTES = 6,
    parameter int W_SECONDS = 6
) (
    input logic clk,
    input logic enable,
    output logic [W_HOURS-1:0] hours,
    output logic [W_MINUTES-1:0] minutes,
    output logic [W_SECONDS-1:0] seconds
);
```

When `enable` is low, all outputs retain their values.

When `enable` is high, `seconds` increments on each rising clock edge. On a seconds rollover (when `seconds` is at `N_SECONDS-1` and `enable` is high), `seconds` wraps to zero and `minutes` increments. Likewise, on a minutes rollover, `minutes` wraps to zero and `hours` increments; on an hours rollover, `hours` wraps to zero.

Rationale We introduced parameters for testing, while deliberately keeping the module name specific rather than generic. Smaller parameter values do more than speed up simulation: they make bugs easier to find, since the test bench is shorter and debugging is more manageable. Had we intended this module to be reused as a general cascaded counter, we would have given it a more generic name instead. Because its name is specific, it is fine to instantiate `hms_counter` without overriding any parameters: the defaults already implement exactly what the name promises.

Implementation Notes Follow these conventions in the implementation:

- Declare two `logic` variables named `second_rollover` and `minute_rollover`.
- Instantiate three `up_down_counter` modules called `u_hour`, `u_minute`, and `u_second`. The `u_` prefix, short for unit, marks these as module instances and avoids name clashes with signals.

To eliminate linter warnings, we must use width-adjusted parameter versions. The following contains a subtle bug:

```
// Incorrect!
localparam logic [W_MINUTES-1:0] NMinutes = W_MINUTES'(N_MINUTES);
localparam logic [W_SECONDS-1:0] NSeconds = W_SECONDS'(N_SECONDS);
```

The widths passed to the module are suitable for storing numbers from 0 to $N-1$, but the code above stores N using the same number of bits. In edge cases, such as $N = 2$, it stores 0 instead of 2 due to truncation that you explicitly authorised.

Because the code needs $N-1$ rather than N, the solution is straightforward:

```
localparam logic [W_MINUTES-1:0] MaxMinutes = W_MINUTES'(N_MINUTES - 1);
localparam logic [W_SECONDS-1:0] MaxSeconds = W_SECONDS'(N_SECONDS - 1);
```

Careful bit-width tracking is vital: writing a logic condition like `x == y+1` is dangerous because `y+1` might wrap around to zero. This danger exists in other programming languages too, but it is most apparent in hardware design, where we work with small numbers of bits. The extra effort to be precise with widths costs minutes and can save hours of debugging.

For linting and compilation, include all required files.

```
verilator -Wall --lint-only rtl/hms_counter.sv rtl/up_down_counter.sv
iverilog -Wall -g2012 rtl/hms_counter.sv rtl/up_down_counter.sv tb/
    wave_hms_counter.v
```

rtl > hms_counter.sv > SystemVerilog - Language Support > hms_counter

```
// HMS Counter
// This module implements a Hours-Minutes-Seconds (HMS) counter using three
// up-down counters. The counter increments the seconds, minutes, and hours
// based on the enable signal and rolls over when reaching the maximum values.
//
// The hours, minutes and seconds are `ticked up` by the up_down_counter
// modules which are instantiated for each time unit. The up_down_counter
// modules are the ones that handle the wrap back to 0 when the maximum value
// is reached, as determined by the MAX parameter argument that was passed.
// into it. Up is always up.
// The rollover is handled here, at the Next-State Logic section, which
// checks if the seconds or minutes have reached their maximum values and sets
// the corresponding rollover signals accordingly.
//
// Parameters:
//   N_HOURS:             - Number of hours (default: 24).
//   N_MINUTES:           - Number of minutes (default: 60).
//   N_SECONDS:           - Number of seconds (default: 60).
//   W_HOURS:             - Width of the hours output (default: 5).
//   W_MINUTES:           - Width of the minutes output (default: 6).
//   W_SECONDS:           - Width of the seconds output (default: 6).
//
// Ports:
//   clk:                    - Clock input.
//   enable:                 - When high, the counter is enabled.
//   hours [W_HOURS-1:0]:    - Output for hours   (0 to N_HOURS-1).
//   minutes [W_MINUTES-1:0]: - Output for minutes (0 to N_MINUTES-1).
//   seconds [W_SECONDS-1:0]: - Output for seconds (0 to N_SECONDS-1).
//

`timescale 1ns / 1ps

module hms_counter #(
    parameter int N_HOURS   = 24,  // number of hours
    parameter int N_MINUTES = 60,  // number of minutes
    parameter int N_SECONDS = 60,  // number of seconds

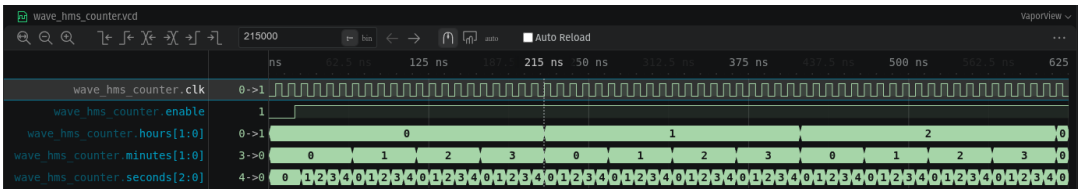
    // Output Port Widths
    parameter int W_HOURS   = 5,  // bit length 5'b for hours (0-23)
    parameter int W_MINUTES = 6,  // bit length 6'b for minutes (0-59)
    parameter int W_SECONDS = 6   // bit length 6'b for seconds (0-59)
) (
    input  logic                clk,
    input  logic                enable,
    output logic [  W_HOURS-1:0] hours,    // = W_HOURS'(0),
    output logic [W_MINUTES-1:0] minutes,  // = W_MINUTES'(0),
    output logic [W_SECONDS-1:0] seconds   // = W_SECONDS'(0)
);

```

rtl > hms_counter.sv > ...

```
47      output logic [W_SECONDS-1:0] seconds   // = W_SECONDS'(0)
48  );
49
50      // Declare Local Parameters and Signals
51      localparam logic [W_MINUTES-1:0] MaxMinutes = W_MINUTES'(N_MINUTES - 1);
52      localparam logic [W_SECONDS-1:0] MaxSeconds = W_SECONDS'(N_SECONDS - 1);
53      logic second_rollover, minute_rollover;
54
55      // State Logic
56      up_down_counter #(
57          .MAX  (N_HOURS - 1),
58          .WIDTH(W_HOURS)
59      ) u_hour (
60          .clk(clk),
61          .enable(minute_rollover),
62          .up(1'b1),
63          .count(hours)
64      );
65
66      up_down_counter #(
67          .MAX  (N_MINUTES - 1),
68          .WIDTH(W_MINUTES)
69      ) u_minute (
70          .clk(clk),
71          .enable(second_rollover),
72          .up(1'b1),
73          .count(minutes)
74      );
75
76      up_down_counter #(
77          .MAX  (N_SECONDS - 1),
78          .WIDTH(W_SECONDS)
79      ) u_second (
80          .clk(clk),
81          .enable(enable),
82          .up(1'b1),
83          .count(seconds)
84      );
85
86      // Next-State Logic
87      always_comb begin
88        if (minutes == MaxMinutes && seconds == MaxSeconds) minute_rollover = 1'b1;
89        else minute_rollover = 1'b0;
90        if (seconds == MaxSeconds) second_rollover = 1'b1;
91        else second_rollover = 1'b0;
92      end
93
94  endmodule
95
```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only rtl/hms_counter.sv rtl/up_down_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/hms_counter.sv rtl/up_down_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/hms_counter.sv rtl/up_down_counter.sv tb/wave_hms_counter.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_hms_counter.vcd opened for output.
tb/wave_hms_counter.v:39: $finish called at 620000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_hms_counter.py
========================= test session starts ==========================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 3 items

tests/test_hms_counter.py ...                                    [100%]

========================== 3 passed in 0.97s ===========================
root@4034a70d84cc:/workspace#
```

6.5. Modulo N Counter

```
module mod_n_counter #(
    parameter int N = 4,
    parameter int WIDTH = 2
) (
    input logic clk,
    input logic rst,
    input logic enable,
    output logic [WIDTH-1:0] count
);
```

The output `count` is initially zero.

On a rising clock edge, `rst` takes priority over `enable`: a high `rst` clears `count` regardless of `enable`.

If only `enable` is high, `count` advances. When it reaches $N - 1$, the next enabled edge wraps it back to zero; otherwise it increments by one.

Rationale Even if `up_down_counter` had a reset pin, a dedicated module with a minimal interface makes the design intent clearer than one with several pins tied high or low.

Implementation Notes Refer to your previous modules for the techniques needed to suppress linter warnings.

It is customary[2] to place reset logic inside the flip-flop block rather than in the next-state logic. Use `always_ff @(posedge clk)` rather than `always @(posedge clk)`; the linter will not flag the latter, but `always_ff` makes the intent explicit.

```
always_ff @(posedge clk)
  if (rst) count <= '0;
  else if (enable) count <= next_count;
```

[2]This mirrors real flip-flops, many of which have a dedicated reset pin.

rtl > mod_n_counter.sv > ...

```sv
// Modulo N Counter
// This module implements a modulo N counter that counts from 0 to N-1 and wraps
// around to 0 when it reaches N. The counter can be reset asynchronously and is
// enabled by an input signal.
//
// Parameters:
//   N      - Maximum count value (default: 4).
//   WIDTH  - Width of the count output (default: 2).
//
// Ports:
//   clk    - Clock input.
//   rst    - Asynchronous reset input.
//   enable - When high, the counter is enabled.
//   count  - Output count value (0 to N-1).
//

`timescale 1ns / 1ps

module mod_n_counter #(
    parameter int N = 4,
    parameter int WIDTH = 2
) (
    input logic clk,
    input logic rst,
    input logic enable,
    output logic [WIDTH-1:0] count = WIDTH'(0)
);

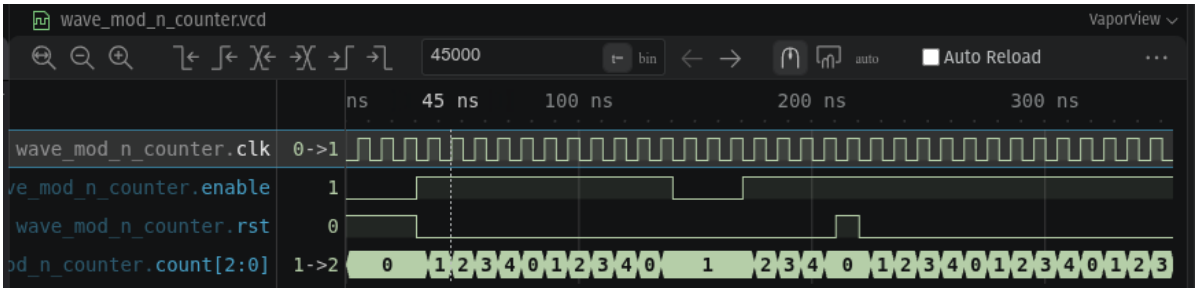
  // Declare local parameters and signals
  // Cast input `N` parameter to local `Max` parameter
  localparam logic [WIDTH-1:0] Max = WIDTH'(N - 1);
  logic [WIDTH-1:0] next_count;

  // State, and Output Logic
  always_ff @(posedge clk) begin
    if (rst) count <= WIDTH'(0);  // Asynchronous reset to 0
    else if (enable) count <= next_count;
  end

  // Next-State Logic
  always_comb begin
    if (count < Max) next_count = count + WIDTH'(1);
    else next_count = WIDTH'(0);  // Wrap to 0 when Max
  end

endmodule
```

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only rtl/mod_n_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/mod_n_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/mod_n_counter.sv tb/wave_mod_n_counter.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_mod_n_counter.vcd opened for output.
tb/wave_mod_n_counter.v:47: $finish called at 350000 (1ps)
root@4034a70d84cc:/workspace#
```




```
root@4034a70d84cc:/workspace# pytest tests/test_mod_n_counter.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 2 items

tests/test_mod_n_counter.py ..                                           [100%]

============================== 2 passed in 0.65s ===============================
root@4034a70d84cc:/workspace#
```

6.6. Restartable Rate Generator

```
module restartable_rate_generator #(
    parameter int CYCLE_COUNT = 2
) (
    input  logic clk,
    input  logic run,
    output logic tick
);
```

When `run` is low, `tick` goes low.

When `run` is high:

- After `run` has been high for exactly `CYCLE_COUNT-1` rising edges, `tick` is high for exactly one clock cycle.
- Thereafter, after each period of `run` having remained high for `CYCLE_COUNT` rising edges, `tick` is again high for exactly one clock cycle.

The edge case `CYCLE_COUNT = 1` is supported: `tick` follows `run` directly.

The module is to be implemented as a Moore FSM.

Rationale This module drives the watch's one-second timekeeping and is reused during time editing, where holding a button causes repeated increments or decrements. Supporting `CYCLE_COUNT = 1` ensures the design remains functional when the watch is configured to tick every clock cycle, which is useful for debugging.

There is no functional reason for this module to be a Moore FSM; the requirement is imposed purely as a learning opportunity.

Implementation Notes There is no elegant way to write this module directly: it is just complex enough that a direct implementation becomes awkward. Instantiating a `mod_n_counter` avoids this, keeping the higher-level logic uncluttered.

In a Moore FSM, outputs depend only on state. Since `tick` must be low whenever `run` is sampled low, `run` must be captured as part of the state.

For `CYCLE_COUNT > 1`, the remaining state is a counter with modulus `CYCLE_COUNT`. Its width is an internal concern — it does not affect any port — and must be computed internally rather than imposed on the instantiating module.

```
localparam int CountWidth = $clog2(CYCLE_COUNT);
```

Report 5

Explain how `localparam CounterWidth = $clog2(CYCLE_COUNT);` works.

CYCLE_COUNT is a (unsigned) integer of width 32
$clog2(number) returns the number of bits required cover the number in a binary string

When `CYCLE_COUNT = 1`, `CounterWidth` is zero, so no counter is needed; a `generate` statement handles this by conditionally omitting the instantiation. Blocks inside a `generate` statement must be named — here `g_general` and `g_special` — or the linter will warn.

```
generate
  if (CYCLE_COUNT > 1) begin : g_general
    // Put code for the general case here,
    // including counter instantiation
  end else begin : g_special
    // Put code for the special case here
  end
endgenerate
```

With readability in mind, we start with the high-level structure:

```
// Becomes high at the end of each cycle
logic tick_qualifier;

logic running = 1'b0;
always_ff @(posedge clk) running <= run;

assign tick = running && tick_qualifier;
```

It remains for the `generate` block to provide `tick_qualifier`. The special case is straightforward:

```
begin : g_special
  assign tick_qualifier = 1'b1;
end
```

A template for the general case follows, with the logic kept separate from the module instantiation.

```
begin : g_general
  localparam int CountWidth = $clog2(CYCLE_COUNT);

  logic rst_count;
  logic enable_count;
  logic [CountWidth-1:0] count;
  mod_n_counter #(
      .N(CYCLE_COUNT),
      .WIDTH(CountWidth)
  ) u_count (
      .clk(clk),
      .rst(rst_count),
      .enable(enable_count),
      .count(count)
  );

  assign rst_count = ...;        // Fill in
  assign enable_count = ...;     // Fill in

  assign tick_qualifier = ...;  // Fill in
end
```

To ensure the correct bit width, use `CountWidth'(CYCLE_COUNT - 1)` when needed.

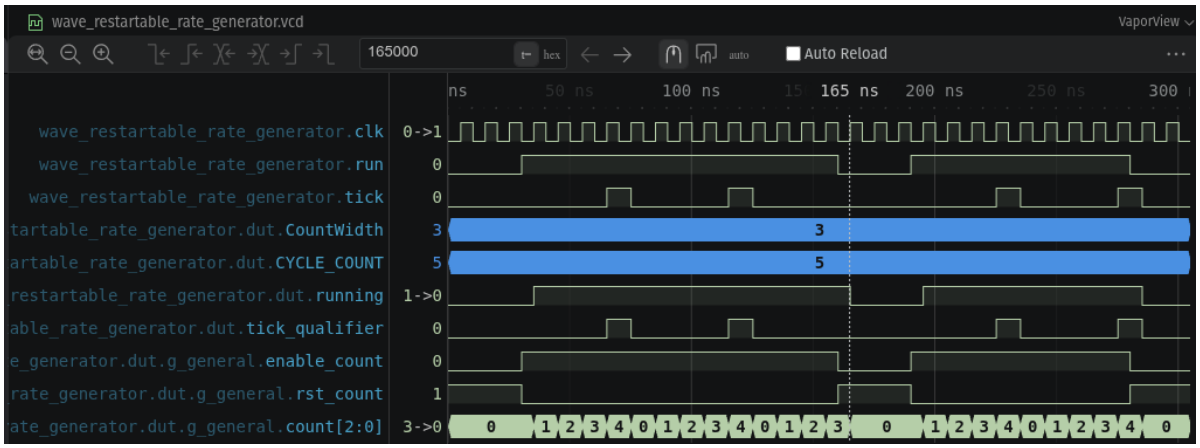
Report 6

In addition to running the provided test bench, write your own test bench verifying correct behaviour when `CYCLE_COUNT = 1`.

Note The module's behaviour may appear asymmetric: the initial delay is `CYCLE_COUNT-1` rising edges, whereas subsequent ticks repeat every `CYCLE_COUNT` rising edges. However, if `run` is synchronised to the clock (i.e., it transitions just after a rising edge, as it would if driven from a flip-flop), the initial delay is also `CYCLE_COUNT` clock cycles, matching the repeat rate.


rtl > restartable_rate_generator.sv > SystemVerilog - Language Support > restartable_rate_generator
// Restartable Rate Generator
// Generates a Mod-N counter that produces a tick at a specified rate when
// enabled. The rate is defined by the CYCLE_COUNT parameter. The generated
// Mod_CYCLE_COUNT counter can be restarted by deasserting and reasserting
// the run signal.
//
// Parameters:
//   CYCLE_COUNT: - Number of clock cycles per tick (default: 2).
//
// Ports:
//   clk: - Clock input.
//   run: - When high, the generated Mod_CYCLE_COUNT instant is
//   enabled and produces ticks after successfully reaching
//   each full cycle.
//   tick: - Output tick signal, high when both run is high and the
//   tick_qualifier is high.
//
// Local Variables:
//   CountWidth: - Bit width of parameter CYCLE_COUNT.
//   rst_count: - Reset signal for the generated `mod_n_counter` counter.
//   enable_count: - Enable signal for the generated `mod_n_counter` counter.
//   count: - Count output of the generated `mod_n_counter` counter.
//   tick_qualifier:- High when the generated `mod_n_counter` counter reaches
//   its maximum value.
//   running: - Indicates whether the rate generator is enabled based
//   on input signal `run`.
`timescale 1ns / 1ps
module restartable_rate_generator #(
parameter int CYCLE_COUNT = 2
) (
input logic clk,
input logic run,
output logic tick
);
// Declare Local Parameters and Signals
// Get required bit width for the counter based on CYCLE_COUNT
localparam int CountWidth = $clog2(CYCLE_COUNT);
// Becomes high at the end of each cycle
logic tick_qualifier;
generate
if (CYCLE_COUNT > 1) begin : g_general
logic rst_count;
logic enable_count;
logic [CountWidth-1:0] count;
.N(CYCLE_COUNT),
.WIDTH(CountWidth)
) u_count (
.clk(clk),
.rst(rst_count),
.enable(enable_count),
.count(count)
);
assign rst_count = !run;
assign enable_count = run;
assign tick_qualifier = (count == CountWidth'(CYCLE_COUNT - 1));
end else begin : g_special
assign tick_qualifier = 1'b1;
end
endgenerate
// Running state logic
logic running = 1'b0;
always_ff @(posedge clk) running <= run;
// Output Logic
assign tick = running && tick_qualifier;
endmodule

```
root@4034a70d84cc:/workspace# verilator -Wall --lint-only rtl/restartable_rate_generator.sv rtl/mod_n_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/restartable_rate_generator.sv rtl/mod_n_counter.sv
root@4034a70d84cc:/workspace# iverilog -Wall -g2012 rtl/restartable_rate_generator.sv rtl/mod_n_counter.sv tb/wave_restartable_rate_generator.v
root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_restartable_rate_generator.vcd opened for output.
tb/wave_restartable_rate_generator.v:33: $finish called at 300000 (1ps)
root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_restartable_rate_generator.py
================================== test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 6 items

tests/test_restartable_rate_generator.py ......                                  [100%]

=================================== 6 passed in 1.77s ===================================
(venv) root@4034a70d84cc:/workspace#
```

6.7. Variable-Speed Time Display

```
module top_time_display_v1 #(
    parameter int CYCLES_PER_SECOND = 50_000_000
) (
    input logic CLOCK_50,
    input logic [1:0] SW,
    output logic [6:0] HEX5,
    output logic [6:0] HEX4,
    output logic [6:0] HEX3,
    output logic [6:0] HEX2,
    output logic [6:0] HEX1,
    output logic [6:0] HEX0
);
```

SW[1:0]	Tick Rate
2'b00	1 Hz
2'b01	25 Hz
2'b10	1 kHz
2'b11	50 MHz

This top-level module for the DE1-SoC board displays the time on the seven-segment displays, initialised to 00:00:00, with the tick rate controlled by `SW[1:0]`.

Rationale The goal is to have a working design ready for the DE1-SoC board when you arrive at the lab. Designing and debugging in VS Code beforehand means it should work first time.

Implementation Notes The final code may be around 100 lines but is straightforward. Begin by instantiating:

- one `hms_counter`;
- three `restartable_rate_generators`, one for each of the slower tick rates;
- three `binary_to_bcds`, one each for hours, minutes and seconds; and
- six `seven_segments`, one for each of the digits.

If it is not immediately obvious what to connect to a port, create a named `logic` variable for it and assign it a value using an `assign` statement at the bottom of the module. This keeps instantiation separate from logic and improves readability, provided the variables are named clearly.

The outputs of `hms_counter` are narrower than the `bin` input of `binary_to_bcd`. Explicitly zero-extend them when making each connection, for example `.bin({2'b0, hours})`.

For each `restartable_rate_generator`, use `CYCLES_PER_SECOND` to derive the `CYCLE_COUNT` value. The default of `50_000_000` matches the DE1-SoC's 50 MHz clock; the test suite overrides it to a smaller value to keep simulation fast.

As the design grows, we can save time by telling `verilator` or `iverilog` to search `rtl/` for any modules it needs. For `iverilog`, we also specify `.sv` as the file extension to search. Note that `-Y .sv` must come before `-y rtl`.

```
verilator -Wall --lint-only -y rtl rtl/top_time_display_v1.sv
iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_top_time_display_v1.v
```

The automated tests check the 1 kHz tick rate but not the slower rates (25 Hz and 1 Hz); these can be verified by hand once the design is on the board.

The simulator may produce warnings labelled `Time: 0`, such as the following.

```
root@a374924768bc:/workspace# ./a.out
VCD info: dumpfile wave_top_time_display_v1.vcd opened for output.
WARNING: rtl/seven_segment.sv:30: value is unhandled for priority or
    unique case statement
          Time: 0 Scope: wave_top_time_display_v1.dut.u_HEX0
```

This is due to a race condition: the **`unique case`** statement in `seven_segment` must have executed before at least one of the `up_down_counter` instances has initialised its output, so some or all bits of `digit` are unknown. Since no case item matches an unknown value, the **`unique`** constraint is correctly reported as violated. There are three solutions.

1. Ignore **`unique case`** warnings labelled `Time: 0`.
2. Add a `$isunknown(digit)` check to handle unknown inputs explicitly.
3. Some simulators allow warnings at time 0 to be suppressed.

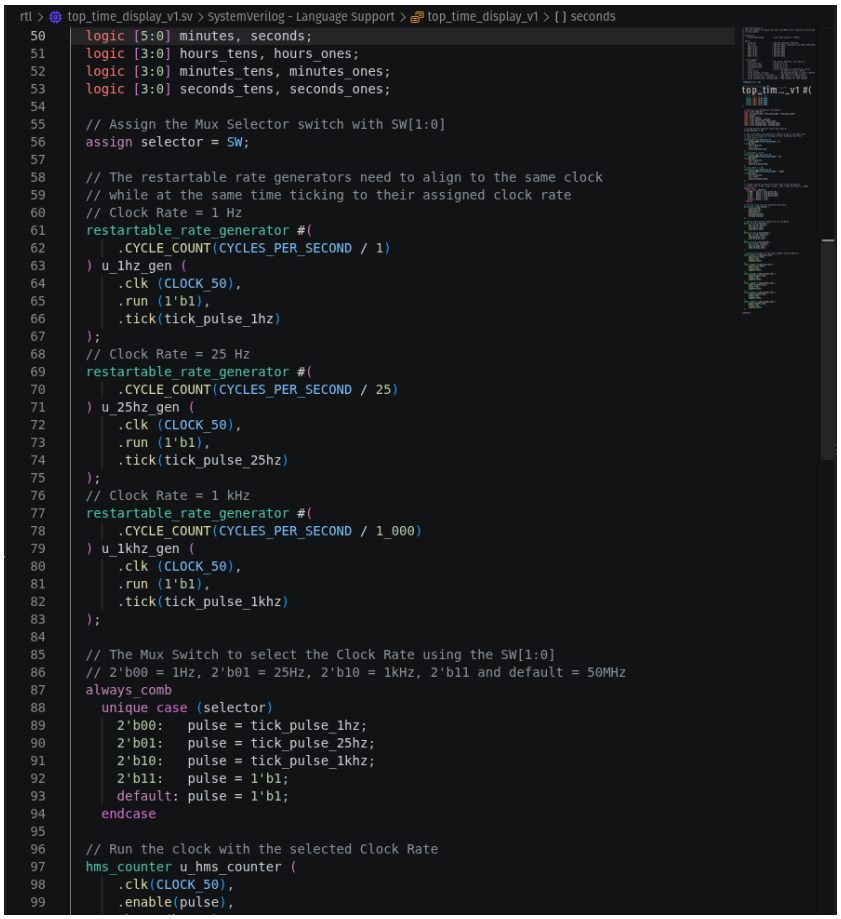
If these warnings truly bother you, add the following to `seven_segment`:

```
if (blank || $isunknown(digit)) segments_active_high = '0;
```

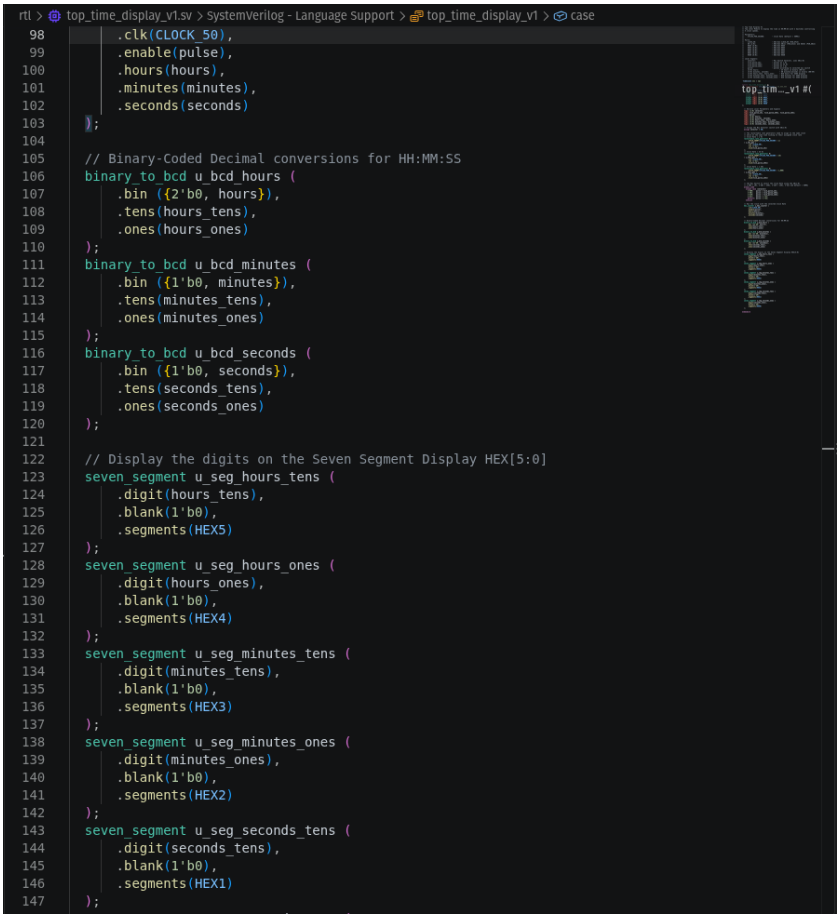
Quartus will ignore this construct, and since the warnings are a simulator quirk, adding extra logic to suppress them is not ideal. The most pragmatic solution is simply to ignore **`unique case`** warnings labelled `Time: 0`.

rtl > top_time_display_v1.sv > ...

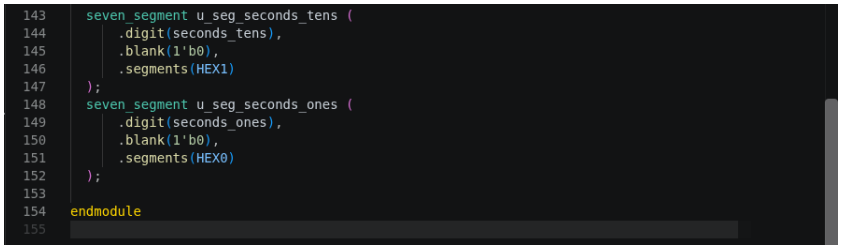
```
1  // Top Time Display V1
2  // Top Level Module to diplay the time in HH:MM:SS with 2 Switches controlling
3  // 4 clock speeds.
4  //
5  // Parameters:
6  //    CYCLES_PER_SECOND:        - Clock Rate (default = 50MHz)
7  //
8  // Ports:
9  //    CLOCK_50:                 - DE1-SoC CLOCK_50 (PIN_AF14)
10 //    SW [1:0]:                 - DE1-SoC SW[1] (PIN_AC12) and SW[0] (PIN_AB12)
11 //    HEX5 [6:0]:               - DE1-SoC HEX5
12 //    HEX4 [6:0]:               - DE1-SoC HEX4
13 //    HEX3 [6:0]:               - DE1-SoC HEX3
14 //    HEX2 [6:0]:               - DE1-SoC HEX2
15 //    HEX1 [6:0]:               - DE1-SoC HEX1
16 //    HEX0 [6:0]:               - DE1-SoC HEX0
17 //
18 // Local Signals:
19 //    selector:                 - Mux Switch Selector, uses SW[1:0]
20 //    tick_pulse_1hz:           - pulses at 1 Hz
21 //    tick_pulse_25hz:          - pulses at 25 Hz
22 //    tick_pulse_1khz:          - pulses at 1 Hz
23 //    pulse:                    - pulses according to selected mux switch
24 //    [4:0] hours:                      - 24 hours in binary (00-23)
25 //    [5:0] minutes, seconds:           - 60 minutes/seconds in binary (00-59)
26 //    [3:0] hours_tens, hours_ones:     - BCD hours for SSEG display
27 //    [3:0] minutes_tens, minutes_ones: - BCD minutes for SSEG display
28 //    [3:0] seconds_tens, seconds_ones: - BCD seconds for SSEG display
29
30 `timescale 1ns / 1ps
31
32 module top_time_display_v1 #(
33     parameter int CYCLES_PER_SECOND = 50_000_000
34 ) (
35     input logic CLOCK_50,
36     input logic [1:0] SW,
37     output logic [6:0] HEX5,
38     output logic [6:0] HEX4,
39     output logic [6:0] HEX3,
40     output logic [6:0] HEX2,
41     output logic [6:0] HEX1,
42     output logic [6:0] HEX0
43 );
44
45   // Declare Local Parameters and Signals
46   logic [1:0] selector;
47   logic tick_pulse_1hz, tick_pulse_25hz, tick_pulse_1khz;
48   logic pulse;
49   logic [4:0] hours;
50   logic [5:0] minutes, seconds;
```



```
rtl > top_time_display_v1.sv > SystemVerilog - Language Support > top_time_display_v1 > { } seconds
50    logic [5:0] minutes, seconds;
51    logic [3:0] hours_tens, hours_ones;
52    logic [3:0] minutes_tens, minutes_ones;
53    logic [3:0] seconds_tens, seconds_ones;
54
55    // Assign the Mux Selector switch with SW[1:0]
56    assign selector = SW;
57
58    // The restartable rate generators need to align to the same clock
59    // while at the same time ticking to their assigned clock rate
60    // Clock Rate = 1 Hz
61    restartable_rate_generator #(
62        .CYCLE_COUNT(CYCLES_PER_SECOND / 1)
63    ) u_1hz_gen (
64        .clk (CLOCK_50),
65        .run (1'b1),
66        .tick(tick_pulse_1hz)
67    );
68    // Clock Rate = 25 Hz
69    restartable_rate_generator #(
70        .CYCLE_COUNT(CYCLES_PER_SECOND / 25)
71    ) u_25hz_gen (
72        .clk (CLOCK_50),
73        .run (1'b1),
74        .tick(tick_pulse_25hz)
75    );
76    // Clock Rate = 1 kHz
77    restartable_rate_generator #(
78        .CYCLE_COUNT(CYCLES_PER_SECOND / 1_000)
79    ) u_1khz_gen (
80        .clk (CLOCK_50),
81        .run (1'b1),
82        .tick(tick_pulse_1khz)
83    );
84
85    // The Mux Switch to select the Clock Rate using the SW[1:0]
86    // 2'b00 = 1Hz, 2'b01 = 25Hz, 2'b10 = 1kHz, 2'b11 and default = 50MHz
87    always_comb
88      unique case (selector)
89        2'b00:   pulse = tick_pulse_1hz;
90        2'b01:   pulse = tick_pulse_25hz;
91        2'b10:   pulse = tick_pulse_1khz;
92        2'b11:   pulse = 1'b1;
93        default: pulse = 1'b1;
94      endcase
95
96    // Run the clock with the selected Clock Rate
97    hms_counter u_hms_counter (
98        .clk(CLOCK_50),
99        .enable(pulse),
```

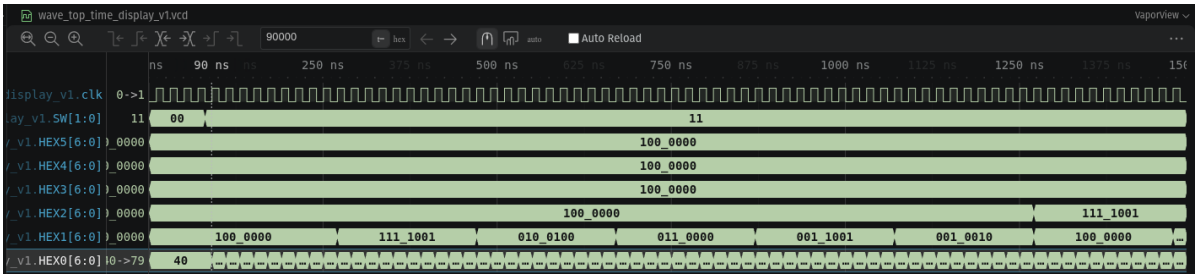


```
rtl > top_time_display_v1.sv > SystemVerilog - Language Support > top_time_display_v1 > case
98        .clk(CLOCK_50),
99        .enable(pulse),
100       .hours(hours),
101       .minutes(minutes),
102       .seconds(seconds)
103   );
104
105   // Binary-Coded Decimal conversions for HH:MM:SS
106   binary_to_bcd u_bcd_hours (
107       .bin ({2'b0, hours}),
108       .tens(hours_tens),
109       .ones(hours_ones)
110   );
111   binary_to_bcd u_bcd_minutes (
112       .bin ({1'b0, minutes}),
113       .tens(minutes_tens),
114       .ones(minutes_ones)
115   );
116   binary_to_bcd u_bcd_seconds (
117       .bin ({1'b0, seconds}),
118       .tens(seconds_tens),
119       .ones(seconds_ones)
120   );
121
122   // Display the digits on the Seven Segment Display HEX[5:0]
123   seven_segment u_seg_hours_tens (
124       .digit(hours_tens),
125       .blank(1'b0),
126       .segments(HEX5)
127   );
128   seven_segment u_seg_hours_ones (
129       .digit(hours_ones),
130       .blank(1'b0),
131       .segments(HEX4)
132   );
133   seven_segment u_seg_minutes_tens (
134       .digit(minutes_tens),
135       .blank(1'b0),
136       .segments(HEX3)
137   );
138   seven_segment u_seg_minutes_ones (
139       .digit(minutes_ones),
140       .blank(1'b0),
141       .segments(HEX2)
142   );
143   seven_segment u_seg_seconds_tens (
144       .digit(seconds_tens),
145       .blank(1'b0),
146       .segments(HEX1)
147   );
```



```
143   seven_segment u_seg_seconds_tens (
144       .digit(seconds_tens),
145       .blank(1'b0),
146       .segments(HEX1)
147   );
148   seven_segment u_seg_seconds_ones (
149       .digit(seconds_ones),
150       .blank(1'b0),
151       .segments(HEX0)
152   );
153
154   endmodule
155
```

```
(venv) root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/top_time_display_v1.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/top_time_display_v1.sv
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/top_time_display_v1.sv:88: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/top_time_display_v1.sv tb/wave_top_time_display_v1.v
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/seven_segment.sv:28: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
rtl/top_time_display_v1.sv:88: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_top_time_display_v1.vcd opened for output.
tb/wave_top_time_display_v1.v:34: $finish called at 1480000 (1ps)
(venv) root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_top_time_display_v1.py
================================== test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_top_time_display_v1.py .                                              [100%]

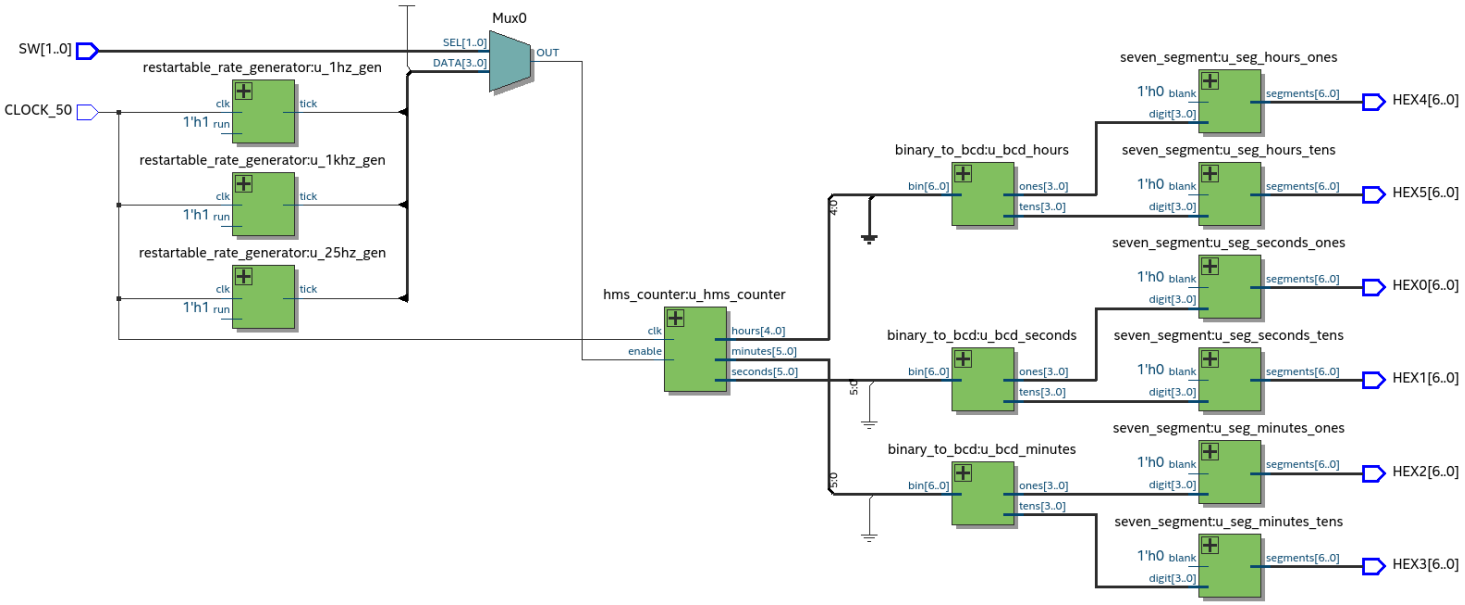
=================================== 1 passed in 0.30s ===================================
(venv) root@4034a70d84cc:/workspace#
```

6.8. Programming the Board

Skip to the next section if you are not in the lab; return here when you are ready to program the board.

Follow these steps to program the DE1-SoC:

1. Create a new Quartus project (e.g., `digital_watch`) with the correct settings for the DE1-SoC board.
2. Set the top-level module to `top_time_display_v1`.
3. Import assignments from `DE1_SoC.qsf`.
4. Create an `.SDC` file specifying the clock period.
5. Copy all your `rtl/*.sv` files into the Quartus project root directory (or under `rtl/` if you prefer).
6. Go to Project » Add/Remove Files and add these files to the project.
7. Compile and program the board.



7. Settable Watch

By the end of this section, we will have a settable watch: time displayed on the DE1-SoC board, incrementing once per second, with the ability to edit the time using the keys. Holding `KEY[3]` for at least one second enters edit mode, with the seconds digits flashing at 2 Hz with an 80% duty cycle. A second press of `KEY[3]` moves to the minutes digits, a third to the hours digits, and a fourth exits edit mode.

In edit mode, `KEY[1]` increments and `KEY[0]` decrements the flashing digits. Holding either key for more than half a second repeats the increment or decrement at 10 Hz.

7.1. Fixed-Frequency Fixed-Duty PWM Generator

```
module pwm_generator #(
    // Number of clock cycles in one PWM period
    parameter int PERIOD_CYCLES = 50_000_000,

    // Number of clock cycles output is high
    parameter int DUTY_CYCLES = 25_000_000
) (
    input  logic clk,
    input  logic rst,
    output logic pwm_out
);
```

The output `pwm_out` is periodic with a period of `PERIOD_CYCLES` clock cycles, high for the first `DUTY_CYCLES` cycles of each period and low for the remainder. When `rst` is asserted, the output restarts from the next rising clock edge.

> **Report 7**
>
> Explain why this module is called a PWM generator.

Rationale During edit mode, the display flashes to indicate which field is being edited. This module provides the PWM signal that drives the flashing.

Implementation Notes Instantiate a `mod_n_counter`. To see its port names while working, right-click `mod_n_counter.sv` in the Explorer panel and select *Open to the Side*.

> **Report 8**
>
> In `mod_n_counter`, `rst` takes priority over `enable`. Comment on the benefit of this design choice.

https://en.wikipedia.org/wiki/Pulse-width_modulation
https://www.sameskydevices.com/blog/pulse-width-modulation-pwm-how-it-works-and-why-its-essential-in-electronics

Pulse-width modulation (**PWM**), also known as **pulse-duration modulation** (**PDM**)[a] or **pulse-length modulation** (**PLM**),[1] is any method of representing a signal as a rectangular wave with a varying duty cycle (and for some methods also a varying period).

PWM facilitates the portion of the period for which the digital signal is active

Summary

Modulation is the application of control over a device or system. Pulse width modulation in particular, is an effective and efficient method used to control power in a wide variety of electronic devices. It allows for precise control and low-power operation when compared to other control technologies and is used across many applications to exert analog-like control using a digital signal.

rtl > pwm_generator.sv > ...

```
// Pulse Width Modulation (PWM) Generator
// Generates a PWM signal with a specified duty and period cycle. `pwm_out`
// signal is high for `DUTY_CYCLES` clock cycles and low for the remainder
// in `PERIOD_CYCLES` clock cycles. Local `count` signal generated by
// Modulo N Counter
//
// Parameters:
//   PERIOD_CYCLES:    - Number of clock cycles in one PWM period
//                       (default: 50_000_000)
//   DUTY_CYCLES:      - Number of clock cycles output is high
//                       (default: 25_000_000)
//
// Ports:
//   clk:              - Clock input
//   rst:              - Asynchronous reset input
//   pwm_out:          - PWM output signal, high when within duty cycle,
//                       low when outside duty cycle
//
// Local Variables:
//   CountWidth:       - Bit width of parameter PERIOD_CYCLES.

`timescale 1ns / 1ps

module pwm_generator #(
    // Number of clock cycles in one PWM period
    parameter int PERIOD_CYCLES = 50_000_000,

    // Number of clock cycles output is high
    parameter int DUTY_CYCLES = 25_000_000
) (
    input  logic clk,
    input  logic rst,
    output logic pwm_out
);

  // Declare local parameters and signals
  // (+1'b1) is needed here to cover edge cases when PERIOD_CYCLES are powers
  // of 2, otherwise there will be integer overflow when DUTY_CYCLES is equal
  // to PERIOD_CYCLES where localparam `DutyCycles` will equal to 0
  localparam int CountWidth = $clog2(PERIOD_CYCLES) + 1'b1;
  localparam logic [CountWidth-1:0] DutyCycles = CountWidth'(DUTY_CYCLES);
  logic [CountWidth-1:0] count;

  // Instaniate Modulo N Counter with N = PERIOD_CYCLES.
  mod_n_counter #(
      .N(PERIOD_CYCLES),
      .WIDTH(CountWidth)
  ) u_period_cycle_count (
      .clk(clk),
      .rst(rst),
      .enable(1'b1),
      .count(count)
  );

  // `pwm_out` is high as for period of DutyCycles within PeriodCycles,
  // past period of DutyCycles, `pwm_out` is low for rest of PeriodCycles.
  always_comb begin
    if (count < DutyCycles) pwm_out = 1'b1;
    else pwm_out = 1'b0;
  end

endmodule
```

```
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile user_wave_pwm_generator.vcd opened for output.
tb/user_wave_pwm_generator.v:29: $finish called at 440000 (1ps)
(venv) root@4034a70d84cc:/workspace# verilator -Wall -y rtl --lint-only rtl/pwm_generator.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/pwm_generator.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_pwm_generator.v
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_pwm_generator.vcd opened for output.
tb/wave_pwm_generator.v:29: $finish called at 410000 (1ps)
(venv) root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_pwm_generator.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 7 items

tests/test_pwm_generator.py .......                                      [100%]

============================== 7 passed in 2.15s ===============================
(venv) root@4034a70d84cc:/workspace#
```

7.2. Rising-Edge Detector

```
module rising_edge_detector (
    input  logic clk,
    input  logic sig_in,
    output logic rise
);
```

The output `rise` is asserted immediately when `sig_in` transitions from low to high, and deasserted as soon as `sig_in` returns low or the next rising clock edge captures the high value, whichever comes first.

Rationale At 50 MHz, a button press spans millions of clock cycles. Without edge detection, a single press of the edit button could trigger millions of mode transitions. A rising edge detector converts the press into a single-cycle pulse, ensuring exactly one transition per press.

Unlike the other modules in this assignment, `rising_edge_detector` is a Mealy FSM — the industry-standard choice for edge detection — as it avoids a one-cycle output delay, is simpler to write, and produces output logic simple enough to be glitch-free.

rtl > rising_edge_detector.sv > ...

```
// Rising Edge Detector
// The output signal `rise` is asserted immediately when input signal `sig_in`
// transitions from low to high, and deasserted when `sig_in` becomes low OR
// when the next rising edge of the clock captures the `sig_in` high signal,
// whichever comes first.
//
// Parameters:
//   None
//
// Ports:
//   clk:       - Clock Input
//   sig_in:    - Input Signal
//   rise:      - Output Signal, a singular pulse within one clock cycle
//
// Local Variables:
//   prev:      - The State that captures the previous value of `sig_in`
//   next_prev: - Transitional state that captures the current
//                value of `sig_in`

`timescale 1ns / 1ps

module rising_edge_detector (
    input  logic clk,
    input  logic sig_in,
    output logic rise
);

  logic prev = 1'b0;  // State
  logic next_prev;

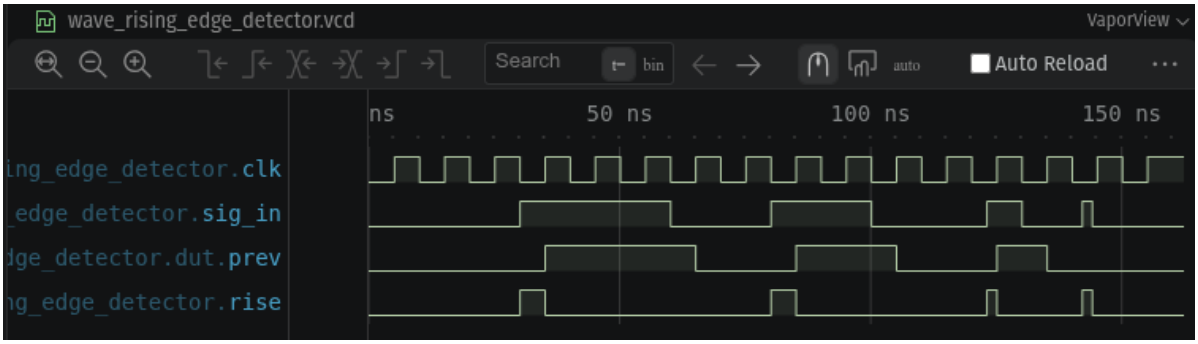
  // Flip-Flip
  always_ff @(posedge clk) begin
    prev <= next_prev;
  end

  // Next-State Logic
  assign next_prev = sig_in;

  // Output Logic
  assign rise = (!prev && sig_in);

endmodule
```

```
(venv) root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/rising_edge_detector.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/rising_edge_detector.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_rising_edge_detector.v
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_rising_edge_detector.vcd opened for output.
tb/wave_rising_edge_detector.v:48: $finish called at 158000 (1ps)
(venv) root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_rising_edge_detector.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_rising_edge_detector.py .                                     [100%]

============================== 1 passed in 0.35s ===============================
(venv) root@4034a70d84cc:/workspace#
```

7.3. Button Hold Detect

```
module button_hold_detect #(
    parameter int HOLD_CYCLES = 50_000_000
) (
    input  logic clk,
    input  logic button,
    output logic held
);
```

The output `held` is asserted on the clock edge at which `button` has been sampled high for `HOLD_CYCLES` consecutive rising edges, and deasserts on the clock edge after `button` goes low.

Rationale Entering edit mode requires the user to hold a button for a minimum duration; this module detects when that threshold has been reached.

Implementation Notes Instantiate a `mod_n_counter` and think carefully about its maximum count. The counter value is the entire FSM state; no additional state variables are needed.

It is more natural to reason about this counter in terms of its maximum value than its modulus `N`. Define `CountMax` explicitly to make the intent clear and avoid off-by-one errors.

```
localparam int CountMax = ... ; // Fill this in
localparam int CountWidth = $clog2(CountMax + 1);

logic count_rst;
logic count_enable;
logic [CountWidth-1:0] count;
mod_n_counter #(
    .N(CountMax + 1),
    .WIDTH(CountWidth)
) u_counter (
    .clk(clk),
    .rst(count_rst),
    .enable(count_enable),
    .count(count)
);
```

From here, derive the next-state and output logic. You may find it convenient for the next-state logic to depend on the output `held`; this is permissible.

> **Report 9**
>
> Explain why the next-state logic may depend on the output `held` without violating the Moore FSM constraint.

Use `CountWidth'(CountMax)` where needed to ensure correct bit widths.

Note As in Section 6.6, the hold duration is exactly `HOLD_CYCLES` clock cycles only if the button is synchronised to the clock.

rtl > button_hold_detect.sv > ...

```
// Button Hold Detect
// When the button is pressed for a specified number of clock cycles, returns
// HIGH as long as the button remains pressed.
//
// Parameters:
//    HOLD_CYCLES: - Number of clock cycles to check against to determine HIGH
//                   (default: 50_000_000)
//
// Ports:
//    clk:         - Clock input
//    button:      - Input button signal
//    held:        - Output signal, HIGH when button is HIGH for at least one
//                 - HOLD_CYCLES clock cycles, LOW otherwise
//
// Local Variables:
//    CountMax:       - To hold parameter value (HOLD_CYCLES - 1)
//    CountWidth:     - Bit width of CountMax
//    count_rst:      - Reset signal for `mod_n_counter` u_counter
//    count_enable:   - Enable signal for `mod_n_counter` u_counter
//    count:          - Count output of `mod_n_counter` u_counter
//    held_qualifier: - HIGH when count reaches CountMax, LOW otherwise

`timescale 1ns / 1ps

module button_hold_detect #(
    parameter int HOLD_CYCLES = 50_000_000
) (
    input  logic clk,
    input  logic button,
    output logic held = 1'b0
);

  localparam int CountMax = HOLD_CYCLES - 1;
  localparam int CountWidth = $clog2(CountMax + 1);

  logic count_rst;
  logic count_enable;
  logic [CountWidth-1:0] count;
  logic held_qualifier;

  mod_n_counter #(
      .N(CountMax + 1),
      .WIDTH(CountWidth)
  ) u_counter (
      .clk(clk),
      .rst(count_rst),
      .enable(count_enable),
      .count(count)
  );

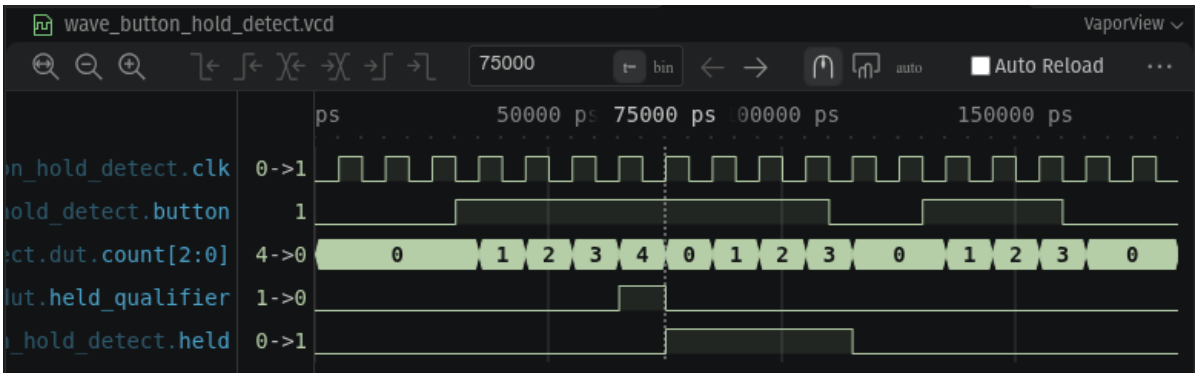
  // Next-State Logic
  assign count_rst = !button;
  assign count_enable = button;
  assign held_qualifier = (count == CountWidth'(CountMax));

  // Flip-Flop, Next-State Logic, and Output Logic
  always_ff @(posedge clk) begin
    if (button && held_qualifier) begin
      held <= 1'b1;
    end else if (button && held) begin
      held <= 1'b1;
    end else begin
      held <= 1'b0;
    end
  end

endmodule

```

```
(venv) root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/button_hold_detect.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/button_hold_detect.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_button_hold_detect.v
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_button_hold_detect.vcd opened for output.
tb/wave_button_hold_detect.v:34: $finish called at 180000 (1ps)
(venv) root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_button_hold_detect.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_button_hold_detect.py .                                       [100%]

============================== 1 passed in 0.36s ===============================
(venv) root@4034a70d84cc:/workspace#
```

Although the state `held` is used as input to determine the next state, the output condition still only depends on the state `held` and not on the `button` input. Thus not violating the Moore FSM constraint

7.4. Button Hold Pulse

This module is provided in full; nevertheless, go through the full development cycle: run `verilator`, `iverilog`, and the automated tests, and include the waveforms in your report.

```
module button_hold_pulse #(
    parameter int HOLD_CYCLES = 50_000_000
) (
    input  logic clk,
    input  logic button,
    output logic pulse
);

  logic held;

  button_hold_detect #(
      .HOLD_CYCLES(HOLD_CYCLES)
  ) u_detect (
      .clk(clk),
      .button(button),
      .held(held)
  );

  rising_edge_detector u_detector (
      .clk(clk),
      .sig_in(held),
      .rise(pulse)
  );

endmodule
```

Report 10

Explain what this module does. Justify your explanation by referring to the generated waveforms, being precise about timing.

Report 11

Explain how `button_hold_pulse` is a Moore FSM despite `rising_edge_detector` being Mealy.

Note Because `rising_edge_detector` is a Mealy FSM, `pulse` asserts on the same clock edge as `held`, rather than one cycle later. A Moore implementation could achieve the same timing by passing `HOLD_CYCLES - 1` to `button_hold_detect`.

rtl > button_hold_pulse.sv > ...

```sv
// Button Hold Pulse
// When the button is pressed for a specified number of clock cycles, returns
// HIGH for only one clock cycle, returns LOW while button remains pressed,
// and when released.
//
// Parameters:
//    HOLD_CYCLES: - Number of clock cycles to check against to determine HIGH
//                   (default: 50_000_000)
//
// Ports:
//    clk:         - Clock input
//    button:      - Input button signal
//    pulse:       - Output signal, HIGH for one clock cycle when button is
//                   HIGH for at least one HOLD_CYCLES clock cycles.
//
// Local Variables:
//    held:        - Wire between output .held from `button_hold_detect` and
//                   input .sig_in of `rising_edge_detector`

`timescale 1ns / 1ps

module button_hold_pulse #(
    parameter int HOLD_CYCLES = 50_000_000
) (
    input  logic clk,
    input  logic button,
    output logic pulse
);

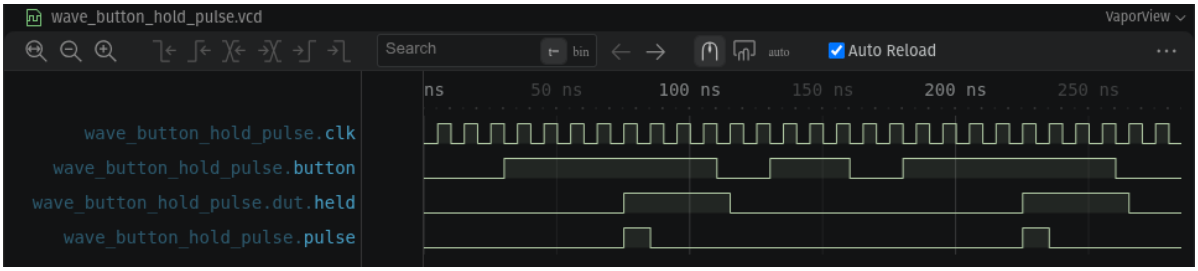
  logic held;

  button_hold_detect #(
      .HOLD_CYCLES(HOLD_CYCLES)
  ) u_detect (
      .clk(clk),
      .button(button),
      .held(held)
  );

  rising_edge_detector u_detector (
      .clk(clk),
      .sig_in(held),
      .rise(pulse)
  );

endmodule
```

```
(venv) root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/button_hold_pulse.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/button_hold_pulse.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_button_hold_pulse.v
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_button_hold_pulse.vcd opened for output.
tb/wave_button_hold_pulse.v:40: $finish called at 280000 (1ps)
(venv) root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_button_hold_pulse.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_button_hold_pulse.py .                                        [100%]

============================== 1 passed in 0.32s ===============================
(venv) root@4034a70d84cc:/workspace#
```

7.5. Button Auto Repeat

```
module button_auto_repeat #(
    parameter int HOLD_CYCLES   = 50_000_000,
    // REPEAT_CYCLES must be smaller than HOLD_CYCLES
    parameter int REPEAT_CYCLES = 5_000_000
) (
    input  logic clk,
    input  logic button,
    output logic pulse
);

  logic rise;
  logic held;
  logic pulse_train;

  assign pulse = rise | (button & pulse_train);
```

The two internal signals, `rise` and `pulse_train`, are derived as follows.

- `button` → `rising_edge_detector` → `rise`
- `button` → `button_hold_detect` → `restartable_rate_generator` → `pulse_train`

A brief press produces an immediate pulse; holding the button produces a pulse train.

The first pulse of the pulse train occurs after `button` has been sampled high for `HOLD_CYCLES` consecutive rising edges; subsequent pulses repeat every `REPEAT_CYCLES` rising edges.

Rationale When editing the watch, a brief press should advance the time by one step; holding the button should produce rapid repeated advances. This module provides both behaviours.

Implementation Notes Instantiate three modules:

- `rising_edge_detector`
- `button_hold_detect`
- `restartable_rate_generator`

Choose their parameters carefully.

Note The rising edge detector is Mealy, so the overall design is Mealy. There is therefore no reason to avoid gating `pulse_train` with `button`: using `button & pulse_train` eliminates the possibility of a spurious pulse one clock cycle after the button is released.

rtl > button_auto_repeat.sv > ...

```sv
// Button Auto Repeat
// When the button is first pressed, a pulse is sent. As long as the button
// remains pressed for at least HOLD_CYCLES, a pulse is sent every
// REPEAT_CYCLES starting from the end of HOLD_CYCLES.
//
// Parameters:
//    HOLD_CYCLES:    - Number of clock cycles to check against to determine
//                      HIGH (default: 50_000_000)
//    REPEAT_CYCLES: - Number of clock cycles to pass to pulse a HIGH
//                      (default: 5_000_000)
//
// Ports:
//    clk:           - Clock input
//    button:        - Input button signal
//    pulse:         - Output signal, HIGH when button is initially pressed
//                      OR while the button is pressed for at least HOLD_CYCLES
//                      send out a HIGH pulse every REPEAT_CYCLES
//
// Local Variables:
//    rise:          - pulse signal for initial press
//    held:          - signal to signify button being held
//    pulse_train:   - waves of pulses repeating every REPEAT_CYCLES as long as
//                      `held` is HIGH


`timescale 1ns / 1ps

module button_auto_repeat #(
    parameter int HOLD_CYCLES   = 50_000_000,
    // REPEAT_CYCLES must be smaller than HOLD_CYCLES
    parameter int REPEAT_CYCLES = 5_000_000
) (
    input  logic clk,
    input  logic button,
    output logic pulse
);

  logic rise;
  logic held;
  logic pulse_train;

  // Sends out a pulse when the button is initially pressed
  rising_edge_detector u_detector (
      .clk(clk),
      .sig_in(button),
      .rise(rise)
  );

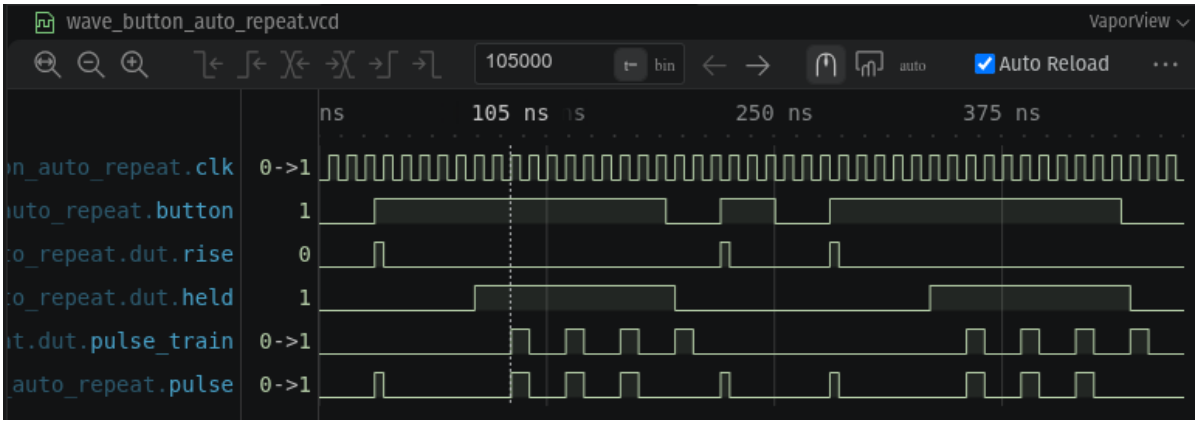
  // Want the pulse to 'tick' upon reaching the HOLD_CYCLE but `pulse_train`
  // only 'ticks' AFTER completing one full REPEAT_CYCLE. So need to offset
  // the starting line by REPEAT_CYCLE
  button_hold_detect #(
      .HOLD_CYCLES(HOLD_CYCLES - REPEAT_CYCLES + 1)
  ) u_detect (
      .clk(clk),
      .button(button),
      .held(held)
  );

  // Repeat Rate = REPEAT_CYCLES
  // pulse_train only ticks when REPEAT_CYCLES is reached only AFTER held
  // is passed in as true
  restartable_rate_generator #(
      .CYCLE_COUNT(REPEAT_CYCLES)
  ) u_1khz_gen (
      .clk (clk),
      .run (held),
      .tick(pulse_train)
  );

  assign pulse = rise | (button & pulse_train);

endmodule
```

```
(venv) root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/button_auto_repeat.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/button_auto_repeat.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_button_auto_repeat.v
(venv) root@4034a70d84cc:/workspace# ./a.out
VCD info: dumpfile wave_button_auto_repeat.vcd opened for output.
tb/wave_button_auto_repeat.v:44: $finish called at 470000 (1ps)
(venv) root@4034a70d84cc:/workspace#
```




```
(venv) root@4034a70d84cc:/workspace# pytest tests/test_button_auto_repeat.py
================================= test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
plugins: metadata-3.1.1, html-4.2.0
collected 1 item

tests/test_button_auto_repeat.py .                                              [100%]

================================== 1 passed in 0.43s ===================================
(venv) root@4034a70d84cc:/workspace#
```

7.6. Arming Latch

```
module arming_latch (
    input  logic clk,
    input  logic arm,
    input  logic disarm,
    output logic armed
);
```

Initially `armed` is low. On each rising clock edge, if `disarm` is high, `armed` is cleared; if only `arm` is high, `armed` is set.

Rationale One instance of this module will track whether the watch is in edit mode.

Note This is simply a flip-flop with synchronous set and clear, where clear takes priority; the port names reflect intended use rather than implementation.